

```
"""
```

```
VA Rating Engine - 38 CFR Part 4
```

```
Implements:
```

- VA combined ratings (whole person math) per § 4.25
- Bilateral factor per § 4.26
- Benefit of the doubt per § 4.3
- TDIU eligibility per § 4.16
- Pyramiding detection per § 4.14

```
"""
```

```
from typing import List, Dict, Tuple
```

```
def va_combined_rating(ratings: List[int]) -> int:
```

```
    """
```

```
    Compute the VA combined rating using the 'whole person' method per 38 CFR § 4
```

```
    Method:
```

1. Sort ratings from highest to lowest.
2. Start with the whole person = 100.
3. For each rating, apply it to the remaining 'efficient whole person'.
4. Subtract each resulting disability from the whole person total.
5. Round the final combined value to the nearest 10% (using 5% as threshold

```
    Returns the final combined rating percentage.
```

```
    """
```

```
    if not ratings:
```

```
        return 0
```

```
    sorted_ratings = sorted(ratings, reverse=True)
```

```
    remaining = 100.0
```

```
    for rating in sorted_ratings:
```

```
        disability = remaining * (rating / 100.0)
```

```
        remaining -= disability
```

```
    combined_raw = 100.0 - remaining
```

```
    # Round to nearest 10 (5 rounds up per VA rules)
```

```
    combined_rounded = round_to_nearest_10(combined_raw)
```

```
    return min(combined_rounded, 100)
```

```
def round_to_nearest_10(value: float) -> int:
```

```

    """Round to nearest 10, with 5 rounding up (VA standard)."""
    remainder = value % 10
    if remainder >= 5:
        return int(value - remainder + 10)
    else:
        return int(value - remainder)

def apply_bilateral_factor(bilateral_pairs: List[Tuple[int, int]]) -> List[int]:
    """
    Apply the bilateral factor per 38 CFR § 4.26.

    When a veteran has a service-connected disability of one extremity
    and also a service-connected disability of the paired extremity,
    a bilateral factor of 10% of the combined value of the bilateral
    conditions is added before combining with all other ratings.

    Args:
        bilateral_pairs: List of (left_rating, right_rating) tuples.

    Returns:
        List of adjusted ratings for bilateral conditions (combined + bilateral b
        ready to be fed into the main combined rating engine with other condition
    """
    adjusted = []

    for left, right in bilateral_pairs:
        # Combine the bilateral pair
        combined_bilateral = va_combined_rating([left, right])
        # Add 10% of that combined value as the bilateral factor
        bilateral_bonus = combined_bilateral * 0.10
        adjusted_combined = combined_bilateral + bilateral_bonus
        adjusted.append(min(round(adjusted_combined), 100))

    return adjusted

def compute_full_rating(
    standalone_ratings: List[int],
    bilateral_pairs: List[Tuple[str, str, int, int]],
) -> Dict:
    """
    Compute the full combined rating for a veteran's claim.

    Args:
        standalone_ratings: List of individual rating percentages for non-bilater
        bilateral_pairs: List of (left_name, right_name, left_rating, right_ratin

```

Returns:

```
    Dict with:
        combined_raw: float - exact combined value before rounding
        combined_final: int - rounded combined rating
        bilateral_adjustments: list of dicts showing bilateral math
        all_ratings_used: list of all ratings fed into final calculation
        tdiu_eligible: bool
        tdiu_reason: str
    """
    all_ratings = list(standalone_ratings)
    bilateral_adjustments = []

    for left_name, right_name, left_r, right_r in bilateral_pairs:
        combined_pair = va_combined_rating([left_r, right_r])
        bonus = combined_pair * 0.10
        adjusted = min(round(combined_pair + bonus), 100)
        all_ratings.append(adjusted)
        bilateral_adjustments.append({
            "left": (left_name, left_r),
            "right": (right_name, right_r),
            "combined_before_factor": combined_pair,
            "bilateral_bonus": round(bonus),
            "adjusted_rating": adjusted,
        })

    # Final combined rating
    remaining = 100.0
    sorted_all = sorted(all_ratings, reverse=True)
    for r in sorted_all:
        remaining -= remaining * (r / 100.0)

    combined_raw = 100.0 - remaining
    combined_final = round_to_nearest_10(combined_raw)
    combined_final = min(combined_final, 100)

    # TDIU eligibility check per 38 CFR § 4.16
    tdiu_eligible, tdiu_reason = check_tdiu(all_ratings, combined_final)

    return {
        "combined_raw": round(combined_raw, 2),
        "combined_final": combined_final,
        "bilateral_adjustments": bilateral_adjustments,
        "all_ratings_used": sorted_all,
        "tdiu_eligible": tdiu_eligible,
        "tdiu_reason": tdiu_reason,
    }
```

```

def check_tdiu(individual_ratings: List[int], combined: int) -> Tuple[bool, str]:
    """
    Check TDIU (Total Disability based on Individual Unemployability) eligibility
    Per 38 CFR § 4.16:

    Scheduling TDIU (§ 4.16(a)):
        - One disability at 60%+ OR
        - Two or more disabilities with one at 40%+ and combined 70%+

    Extrascheduling TDIU (§ 4.16(b)):
        - Veteran is unable to secure or follow substantially gainful occupation
          due to service-connected disabilities, even if not meeting scheduling three
    """
    max_single = max(individual_ratings) if individual_ratings else 0

    # Single-disability TDIU
    if max_single >= 60:
        return True, (
            f"Scheduling TDIU eligible under § 4.16(a): one disability rated at {max_single}% "
            f"(threshold: 60% single disability)."
        )

    # Multi-disability TDIU
    has_40_plus = any(r >= 40 for r in individual_ratings)
    if combined >= 70 and has_40_plus:
        return True, (
            f"Scheduling TDIU eligible under § 4.16(a): combined rating {combined}% "
            f"with at least one disability at 40%+ (thresholds: combined 70%, one "
            f"disability 40%+)."
        )

    # Extrascheduling: flag if combined is 60%+ even if not meeting scheduling three
    if combined >= 60:
        return True, (
            f"Potential TDIU eligibility under § 4.16(b) (extrascheduling): "
            f"combined {combined}%. If your disabilities prevent substantially gainful "
            f"employment, you may qualify regardless of individual rating percentages."
        )

    return False, ""

def estimate_monthly_compensation(combined_rating: int, dependent_count: int = 0)
    """
    Estimate monthly VA disability compensation based on 2024 VA rates.
    Note: Advise veteran that rates are updated annually by Congress.
    """

```

```

"""
# 2024 VA compensation rates (single veteran, no dependents)
BASE_RATES_2024 = {
    10: 171.23,
    20: 338.49,
    30: 524.31,
    40: 755.28,
    50: 1075.16,
    60: 1361.88,
    70: 1716.28,
    80: 1995.01,
    90: 2241.91,
    100: 3737.85,
}

# Approximate dependent additions at 30%+ (varies by rating)
DEPENDENT_ADDITIONS = {
    30: 51,
    40: 60,
    50: 70,
    60: 79,
    70: 88,
    80: 97,
    90: 106,
    100: 115,
}

base = BASE_RATES_2024.get(combined_rating, 0)

# Find nearest dependent addition bracket
dep_key = None
for k in sorted(DEPENDENT_ADDITIONS.keys(), reverse=True):
    if combined_rating >= k:
        dep_key = k
        break

dep_addition = (DEPENDENT_ADDITIONS.get(dep_key, 0) * dependent_count) if dep
total = base + dep_addition

return {
    "combined_rating": combined_rating,
    "base_monthly": round(base, 2),
    "dependent_monthly_addition": round(dep_addition, 2),
    "estimated_total_monthly": round(total, 2),
    "annual_estimate": round(total * 12, 2),
    "disclaimer": "Rates are based on 2024 VA compensation tables and updated
                  "Actual payment depends on your specific dependents, deduct

```

```
}
```

```
def benefit_of_the_doubt(lower_rating: int, higher_rating: int, symptoms: List[str])
    """
    Apply 38 CFR § 4.3 - Resolution of Reasonable Doubt.
    When evidence is in approximate balance, the rating must be
    resolved in favor of the veteran (higher rating).

    Returns recommendation dict.
    """
    return {
        "recommended_rating": higher_rating,
        "applied_rule": "38 CFR § 4.3 - Benefit of the Doubt",
        "rationale": (
            f"Your documented symptoms place you between the {lower_rating}% and "
            f"rating criteria. Under 38 CFR § 4.3, when there is an approximate b "
            f"for and against a higher rating, the claim must be resolved in the "
            f"Recommend documenting the following symptoms to solidify the {high "
            + "; ".join(symptoms)
        ),
    }
```